



Trust Skills, Not Portfolios.

Problem Statement Document

Evidence-led definition of the trust gap in software freelancing

Deliverable 01

Product: Scora Team: Code Luck

Phase: Product Validation & Strategy Date: 27 July 2026

Executive summary

The problem in one sentence

AI can now generate convincing software quickly, so clients cannot reliably distinguish a developer who owns and understands the code from a person who can only prompt, copy, and deliver output they cannot explain, debug, secure, or maintain.

Scora exists to protect the verified developer marketplace from unearned technical claims. A candidate is not verified because code runs or looks polished. Verification requires evidence that the candidate can explain the solution, locate defects, change requirements, defend trade-offs, test edge cases, and identify security and maintainability risks.

Product position: using AI is not an automatic failure. A capable developer may use AI as a tool and still demonstrate ownership. The profile is withheld or marked unverified when the person repeatedly cannot demonstrate the engineering understanding required for the claimed skill level.

1. Problem description

Generative coding tools reduce the effort required to produce plausible repositories, features, proposals, and portfolio samples. This is valuable for real developers, but it also lowers the visible difference between an engineer and an applicant whose capability disappears when the tool is wrong, the requirements change, or production behavior must be investigated.

Traditional freelance marketplaces are optimized for discovery, contracting, reputation, and client satisfaction. Those signals matter, but they do not directly prove who authored a portfolio, whether the freelancer understands submitted code, or whether the person can safely maintain it after delivery. The problem appears before the first contract and may remain hidden until a failure, change request, or security incident.

The evidence does not support a simplistic ban on AI. GitHub reports quality improvements in a controlled study of experienced developers, while security studies and the Stack Overflow survey show inaccurate output, misplaced confidence, and debugging burden in other settings. The reliable distinction is therefore not AI versus no AI; it is verified engineering ownership versus dependence without understanding.

2. Why this problem matters

Stakeholder	Business impact	Human impact
Clients / SMEs	Rework, fragile systems, missed deadlines, security exposure, and costly replacement.	A polished demo can hide the fact that the seller cannot explain or maintain the implementation.
Verified developers	Genuine expertise competes against low-cost generated output and inflated claims.	Engineering judgment, debugging, and ownership are harder to show than a polished AI-generated portfolio.
Early-career developers	A capable newcomer still lacks reviews, while an unskilled applicant can manufacture impressive-looking evidence.	The cold-start problem becomes an evidence-authenticity problem.
Platform	False positives create disputes, low-quality delivery, support cost, and loss of trust.	A verified pool must exclude unsupported skill claims without turning AI usage itself into misconduct.

3. Evidence and market signals

Evidence	Observed signal	Implication for Scora
Accuracy and debugging	Stack Overflow 2025 reports 46% distrust AI accuracy; 66% cite almost-right answers and 45% cite extra time debugging AI-generated code.	Scora tests whether the candidate can detect and repair plausible-but-wrong output.
Security and confidence	Perry et al. found AI-assisted participants wrote less secure code in the tested tasks and were more likely to believe it was secure.	Passing tests and confidence are insufficient; candidates must identify threats, assumptions, and negative cases.
Context matters	GitHub reports gains among experienced developers, while another USENIX study found only a small security difference in its C task.	Scora evaluates the person and evidence in context; it does not classify all AI users as weak.
Hiring signal gap	Research with 32 industry professionals reports difficulty evaluating true ability after GenAI adoption.	A code-grounded interview and live modification provide evidence beyond the submitted artifact.
Market design	Upwork's JSS measures satisfaction and contract outcomes; HackerRank and Codility use structured technical assessment.	Scora joins technical verification to a freelance profile and verified talent pool.

4. Current market gaps

Category	What it does well	Remaining gap
Freelance marketplaces	Discovery, contracts, identity, reviews, outcome history, and reputation.	They mainly measure what happened around previous contracts, not whether current code claims are understood.
Assessment platforms	Consistent tests, realistic tasks, AI policies, interviews, and integrity controls.	Usually serve an employer's hiring event; the evidence is not a persistent identity inside a freelance marketplace.
Curated networks	High-touch screening and client confidence.	Expensive to operate and often communicate only accepted/rejected rather than evidence, confidence, and growth.
Portfolios / Git hosting	Rich artifacts and public activity.	AI can generate convincing artifacts; authorship, contribution, constraints, and code ownership remain uncertain.
Certificates / courses	Structured learning and recognizable completion evidence.	Completion does not prove the candidate can independently explain, modify, debug, secure, and maintain software.

5. Core pain points

Developer pains	Client pains
• Cold start without platform reviews. • AI-generated portfolios can compete with real engineering evidence. • Generic coding puzzles miss architecture and maintainability. • No portable, current proof of skill across opportunities. • No clear way to prove responsible AI use and genuine code ownership.	• Hard to compare developers consistently. • Dependence on AI may become visible only after the contract starts. • Non-technical clients cannot audit architecture claims. • Interviews consume scarce senior-engineer time. • A working demo does not prove debugging or maintenance ability.

6. Why existing solutions are insufficient

- A generated artifact can pass visual inspection. A portfolio or take-home project may look professional even when the submitter cannot reconstruct the reasoning.
- Automated tests validate output, not ownership. Passing cases do not prove the candidate can explain the architecture, find missing cases, or change the solution.
- Reputation is delayed. Reviews may expose a weak freelancer only after clients have already paid the learning cost.
- AI detection is not proof. Detecting tool usage is unreliable and would also punish capable developers who use AI responsibly.
- Proctoring alone measures behavior. It may detect suspicious events, but understanding must be demonstrated through explanation, debugging, and modification.
- A score without evidence can be gamed or misunderstood. Scora needs component evidence, confidence, recency, versioning, and an appeal process.

7. Real-world examples

The following are evidence-informed scenarios, not claimed customer interviews. We use them to define the cases that our primary interviews and product tests will validate.

Scenario	Failure today	Scora response
New graduate with strong projects	No reviews and limited paid history make the profile risky to clients.	A verified, explainable skill profile supplies an alternative first trust signal.
SME founder hiring an API developer	The founder can judge price and portfolio appearance but not security, tests, or architecture.	A client-readable report reduces technical information asymmetry.
Prompt-only applicant	A polished feature is submitted, but the applicant cannot explain data flow, repair a hidden bug, or adapt a requirement.	The assessment fails the understanding and modification gates; no verified badge is issued for that track.
Responsible AI-assisted developer	The developer used AI but reviews the diff, explains the design, finds weak output, adds tests, and completes a live change.	Scora records verified ownership and AI-collaboration evidence rather than penalizing tool use.

Scenario	Failure today	Scora response
Experienced engineer changing stack	Past reputation is strong but does not prove capability in the new technology.	Skill-specific assessments separate transferable engineering strength from stack proficiency.

8. Problem scope

In Phase-1 MVP	Out of initial scope
Software developers only; begin with one or two tracks.	All digital professions or every programming language.
No-AI fundamentals checkpoint plus an AI-allowed engineering task, code review, debugging, and code-specific interview.	Trying to prove that a specific line was written by AI.
Explainable Skill Points and Trust Score with confidence and recency.	A guaranteed or manipulation-proof universal measure of talent.
Withhold verification when required understanding gates are not met; support reassessment and appeal.	Permanent punishment or public accusation of cheating.
Only verified profiles appear in the trusted developer pool for the claimed track.	Banning every developer who uses an AI coding assistant.

9. Assumptions and validation plan

Claim to validate	Test	Pass signal
Understanding gates separate ownership from output-only success.	Compare test-only results with explanation, debugging, and live-change ratings across 50 attempts.	The multi-stage rubric identifies material false positives missed by tests alone.
Capable AI users still pass fairly.	Run equivalent AI-allowed and no-AI evidence paths with senior-engineer calibration.	Responsible AI use does not reduce ratings when understanding and engineering outcomes are equivalent.
Clients value verified ownership above another badge.	Interview 8-12 SME founders / hiring managers with evidence-card prototypes.	At least 60% rank ownership evidence in their top two decision inputs.
Developers accept a strict but transparent gate.	Test policy, sample report, appeal, and reassessment concepts.	Qualified participants understand the rule and perceive the evidence requirements as fair.
Selective review preserves accuracy.	Compare automated confidence thresholds with blinded senior-engineer ratings.	Flagged cases contain materially more disagreement than auto-resolved cases.

References

We selected sources for their direct relevance and authority. Company pages describe current product behavior; surveys and research papers support market and risk claims. Assumptions that still require interviews or field testing are identified in the validation sections.

1. Stack Overflow 2025 Developer Survey - AI. Among respondents answering the accuracy question, 46% distrust AI-tool accuracy versus 33% who trust it. The most reported frustration is output that is almost right, while 45% cite extra time debugging AI-generated code.
<https://survey.stackoverflow.co/2025/ai>
2. GitHub randomized controlled study on Copilot and code quality. A study with 202 developers who had at least five years of experience found improved functionality and modest quality gains with Copilot. AI use alone is therefore not evidence of low skill.
<https://github.blog/news-insights/research/does-github-copilot-improve-code-quality-heres-what-the-data-says/>
3. Do Users Write More Insecure Code with AI Assistants?. In a security-focused user study, participants with an AI assistant produced less secure code and were more likely to believe their code was secure; lower trust and more active engagement correlated with fewer vulnerabilities.
<https://arxiv.org/abs/2211.03622>
4. Lost at C - USENIX Security 2023. A different user study found only a small security impact in its specific C task. This mixed evidence shows that risk depends on the task, user, and verification process rather than AI use alone.
<https://www.usenix.org/conference/usenixsecurity23/presentation/sandoval>
5. METR - Early-2025 AI and experienced developer productivity. A randomized study of experienced open-source developers working in familiar repositories found early-2025 AI tools slowed completion in that setting, despite participants expecting a speedup.
<https://metr.org/blog/2025-07-10-early-2025-ai-experienced-os-dev-study/>
6. Generative AI code tools and software-engineer hiring. A study of 32 industry professionals reports that GenAI complicates evaluation of candidates' true abilities, while many participants value candidates who can demonstrate effective and informed AI use.
<https://arxiv.org/abs/2409.00875>
7. OWASP Secure Coding with AI. OWASP recommends independent verification, full-diff review, adversarial testing, dependency checks, human accountability, and sandboxed execution for AI-assisted development.
https://cheatsheetseries.owasp.org/cheatsheets/Secure_Coding_with_AI_Cheat_Sheet.html
8. Upwork Job Success Score. Upwork's Job Success Score reflects client satisfaction, contract outcomes, and long-term client relationships. It is a reputation signal rather than a direct test of code ownership or current engineering understanding.
<https://support.upwork.com/hc/en-us/articles/38437362753171-What-is-a-Job-Success-Score>
9. Upwork - Build a complete profile. Upwork emphasizes work history, certifications, availability, achievements, keywords, and portfolios as profile trust inputs.
<https://support.upwork.com/hc/en-us/articles/34924882793107-Build-a-100-complete-profile>
10. HackerRank Screen - AI-era skills and integrity. HackerRank separates fundamentals without AI, ability to work with AI, and AI concepts, while adding identity and assessment-integrity controls. This validates multi-mode assessment in the AI era.
<https://www.hackerrank.com/products/screen>
11. Codility Task Library. Codility combines real-life skills, problem solving, and technical knowledge tasks, supporting a rounded assessment rather than a single puzzle score.
<https://support.codility.com/hc/en-us/articles/360043827713-The-Codility-Task-Library>
12. Docker Engine security. Docker documents namespaces, cgroups, capabilities, daemon attack surface, and hardening controls relevant to isolated code execution.
<https://docs.docker.com/engine/security/>